

React Native Developer - Take-Home Assignment

Objective

Build a product listing app with search, cart, offline support, and a WebView screen. This assignment mirrors real problems you'd encounter on our app — including a few details that are easy to miss. We're looking for how you think, not just whether the app runs.

1. Product Listing Screen

- Fetch products from the DummyJSON API (<https://dummyjson.com/products>), which supports pagination — use it rather than fetching everything at once.
- Implement infinite scroll (not a “Load More” button).
- Add a search bar that queries the API's search endpoint (`/products/search?q=`), properly debounced so you're not firing a request on every keystroke.

2. Product Detail Screen

- Show full product info, with an image carousel if the product has multiple images.
- The API returns a `discountPercentage` field — calculate and display the discounted price.

3. Cart Functionality

- Use Context API, Redux, or Zustand — your choice, but be ready to justify the trade-off.

4. Persistence

- Persist the cart to `AsyncStorage`.

5. WebView Integration

- Add a “Return Policy” link/button on the product detail screen that opens a WebView (any static URL works, e.g. a public doc or sample policy page).

6. Mock Analytics / Event Logging

- Log at least four events: `product_viewed`, `add_to_cart`, `search_performed`, `app_backgrounded` — with relevant metadata (product ID, timestamp, etc). Console logging or an in-app log screen is fine; no real analytics SDK required.

7. Bonus (optional)

- Handle the API being temporarily down — implement retry with exponential backoff.
- Add pull-to-refresh that doesn't conflict with infinite scroll's pagination logic.
- Write one test that specifically covers the race-condition search fix.
- Persist and support dark mode.

Submission Guidelines

1. Share a GitHub repo link (public, or invite access — details below).
2. Include a README covering: setup/run instructions, any assumptions or trade-offs you made, and what you'd improve with more time.
3. A short 2–3 minute screen recording (Loom or similar) walking through the app.

What We're Evaluating

- Whether you noticed the trickier requirements or just built the happy path.
- How you handle async/race conditions and edge cases (loading, empty, error states).
- Re-render awareness and general performance instincts.
- Code organization, readability, and Git commit hygiene.
- Ability to explain your reasoning and trade-offs — we may ask you to walk through your solution in a follow-up conversation.

Questions? Reach out to the recruiting contact who shared this document with you. We're excited to see what you build!